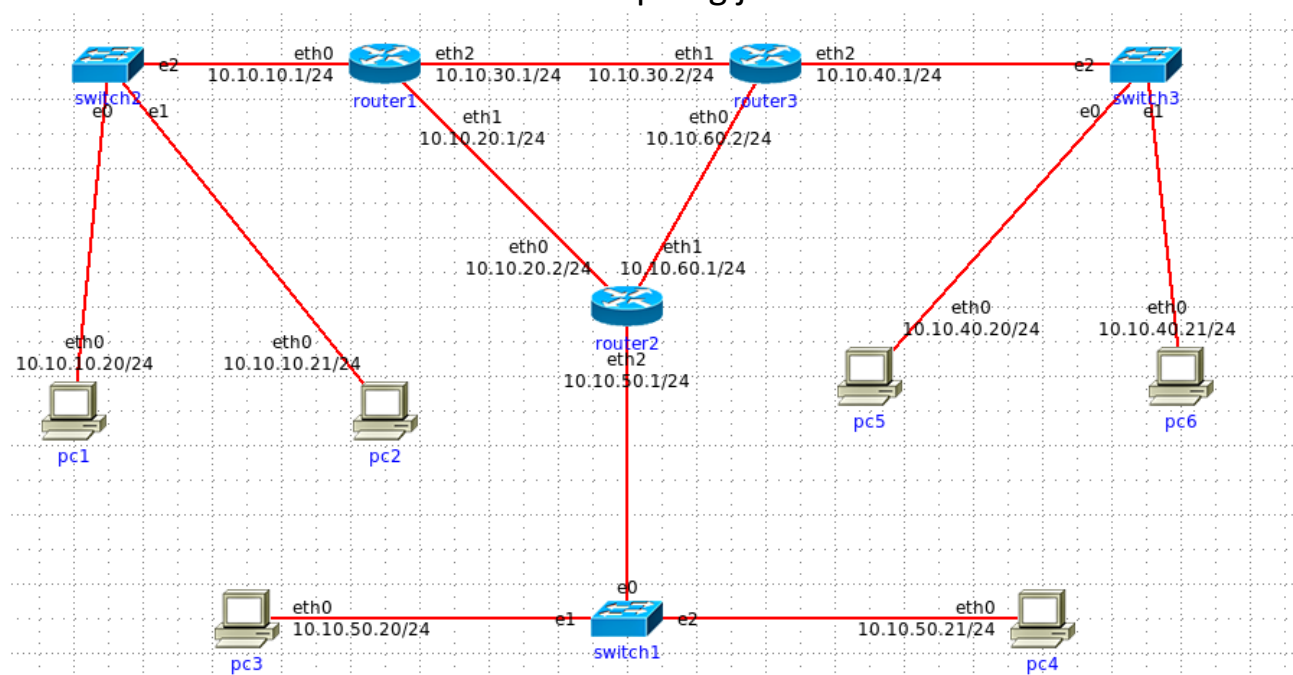


KOMUNIKACIJSKE MREŽE – 2. LABORATORIJSKA VJEŽBA PRIPREMA

Zadatak 22. (Vlastito izrađena topologija) U emulatoru/simulatoru IMUNES konstruirajte mrežu koja sadrži tri podmreže povezane usmjeriteljima (Slika 3.16). Konfigurirajte statičko usmjeravanje između svih podmreža (dakle, bez korištenja protokola za usmjeravanje). Sučeljima računala i usmjeritelja dodijelite IP-adrese iz raspona 10.10.10.0/24, 10.10.20.0/24, 10.10.30.0/24, 10.10.40.0/24, 10.10.50.0/24 i 10.10.60.0/24. Ispišite tablice usmjeravanja svih računala i usmjeritelja.

Odgovor: Prilažem slike vlastito izrađene topologije i tablica usmjeravanja svih računala i usmjeritelja. Tablice usmjeravanja sam dobio izvedbom naredbe *netstat -4 -rn* u ljusci *bash* za pojedinačni element.

Izrađena topologija



Pc 1

```
[root@pc1 /]# netstat -4 -rn
Routing tables

Internet:
Destination      Gateway         Flags         Netif Expire
default          10.10.10.1     UGS          eth0
10.10.10.0/24    link#2        U           eth0
10.10.10.20     link#2        UHS          lo0
127.0.0.1       link#1        UH           lo0
```

Pc 2

```
[root@pc2 /]# netstat -4 -rh
Routing tables

Internet:
Destination      Gateway          Flags    Netif Expire
default          10.10.10.1      UGS      eth0
10.10.10.0/24    link#2         U        eth0
10.10.10.21     link#2         UHS      lo0
localhost        link#1         UH       lo0
```

Pc 3

```
[root@pc3 /]# netstat -4 -rh
Routing tables

Internet:
Destination      Gateway          Flags    Netif Expire
default          10.10.50.1      UGS      eth0
10.10.50.0/24    link#2         U        eth0
10.10.50.20     link#2         UHS      lo0
localhost        link#1         UH       lo0
```

Pc 4

```
[root@pc4 /]# netstat -4 -rh
Routing tables

Internet:
Destination      Gateway          Flags    Netif Expire
default          10.10.50.1      UGS      eth0
10.10.50.0/24    link#2         U        eth0
10.10.50.21     link#2         UHS      lo0
localhost        link#1         UH       lo0
```

Pc 5

```
[root@pc5 /]# netstat -4 -rh
Routing tables

Internet:
Destination      Gateway          Flags    Netif Expire
default          10.10.40.1      UGS      eth0
10.10.40.0/24    link#2         U        eth0
10.10.40.20     link#2         UHS      lo0
localhost        link#1         UH       lo0
```

Pc 6

```
[root@pc6 /]# netstat -4 -rh
Routing tables

Internet:
Destination      Gateway          Flags    Netif Expire
default          10.10.40.1      UGS      eth0
10.10.40.0/24    link#2         U        eth0
10.10.40.21     link#2         UHS      lo0
localhost        link#1         UH       lo0
```

Router 1

```
[root@router1 /]# netstat -4 -rh
Routing tables

Internet:
Destination      Gateway          Flags    Netif Expire
10.10.10.0/24     link#2          U        eth0
10.10.10.1       link#2          UHS      lo0
10.10.20.0/24     link#3          U        eth1
10.10.20.1       link#3          UHS      lo0
10.10.30.0/24     link#4          U        eth2
10.10.30.1       link#4          UHS      lo0
10.10.40.0/24     10.10.30.2      UGS      eth2
10.10.50.0/24     10.10.20.2      UGS      eth1
localhost        link#1          UH       lo0
```

Router 2

```
[root@router2 /]# netstat -4 -rh
Routing tables

Internet:
Destination      Gateway          Flags    Netif Expire
10.10.10.0/24     10.10.20.1      UGS      eth0
10.10.20.0/24     link#2          U        eth0
10.10.20.2       link#2          UHS      lo0
10.10.40.0/24     10.10.60.2      UGS      eth1
10.10.50.0/24     link#4          U        eth2
10.10.50.1       link#4          UHS      lo0
10.10.60.0/24     link#3          U        eth1
10.10.60.1       link#3          UHS      lo0
localhost        link#1          UH       lo0
```

Router 3

```
[root@router3 /]# netstat -4 -rh
Routing tables

Internet:
Destination      Gateway          Flags    Netif Expire
10.10.10.0/24     10.10.30.1      UGS      eth1
10.10.30.0/24     link#3          U        eth1
10.10.30.2       link#3          UHS      lo0
10.10.40.0/24     link#4          U        eth2
10.10.40.1       link#4          UHS      lo0
10.10.50.0/24     10.10.60.1      UGS      eth0
10.10.60.0/24     link#2          U        eth0
10.10.60.2       link#2          UHS      lo0
localhost        link#1          UH       lo0
```

Zadatak 23. (Vlastito izrađena topologija) U emulatoru/simulatoru IMUNES konstruirajte mrežu koja sadrži tri podmreže povezane usmjeriteljima (Slika 3.16). Konfigurirajte statičko usmjeravanje (dakle, bez korištenja protokola za usmjeravanje) tako da dođe do petlje u usmjeravanju. Sučeljima računala i usmjeritelja dodijelite IP-adrese iz raspona 10.10.10.0/24, 10.10.20.0/24, 10.10.30.0/24, 10.10.40.0/24, 10.10.50.0/24 i 10.10.60.0/24. Ispišite tablice usmjeravanja svih računala i usmjeritelja. Pomoću alata Wireshark

utvrdite što se tada događa s paketima koji "uđu" u petlju te komentirajte svoja zapažanja.

Odgovor: Tablice usmjerenja za računala (PC) ostaju iste zbog toga što nisu direktno povezane s petljom. Međutim, tablice usmjerenja usmjerivača (router) se mijenjaju pošto datagrami zapinju u petlji.

Router 1

```
[root@router1 /]# netstat -4 -rh
Routing tables

Internet:
Destination      Gateway          Flags    Netif Expire
10.10.10.0/24     link#2           U        eth0
10.10.10.1       link#2           UHS      lo0
10.10.20.0/24     link#3           U        eth1
10.10.20.1       link#3           UHS      lo0
10.10.30.0/24     link#4           U        eth2
10.10.30.1       link#4           UHS      lo0
10.10.40.0/24     10.10.20.2       UGS      eth1
10.10.50.0/24     10.10.30.2       UGS      eth2
localhost        link#1           UH       lo0
```

Router 2

```
[root@router2 /]# netstat -4 -rh
Routing tables

Internet:
Destination      Gateway          Flags    Netif Expire
10.10.10.0/24     10.10.60.2       UGS      eth1
10.10.20.0/24     link#2           U        eth0
10.10.20.2       link#2           UHS      lo0
10.10.40.0/24     10.10.20.1       UGS      eth0
10.10.50.0/24     link#4           U        eth2
10.10.50.1       link#4           UHS      lo0
10.10.60.0/24     link#3           U        eth1
10.10.60.1       link#3           UHS      lo0
localhost        link#1           UH       lo0
```

Router 3

```
[root@router3 /]# netstat -4 -rh
Routing tables

Internet:
Destination      Gateway          Flags    Netif Expire
10.10.10.0/24     10.10.60.1       UGS      eth0
10.10.30.0/24     link#3           U        eth1
10.10.30.2       link#3           UHS      lo0
10.10.40.0/24     link#4           U        eth2
10.10.40.1       link#4           UHS      lo0
10.10.50.0/24     10.10.30.1       UGS      eth1
10.10.60.0/24     link#2           U        eth0
10.10.60.2       link#2           UHS      lo0
localhost        link#1           UH       lo0
```

Kako se pakete nalaze u petlji između dva rutera oni nikada neće doći na svoje odredište te će biti odbačeni nakon što njihov TTL dosegne vrijednost nule.

Zadatak 24. (Topologija RIP/RIP1.imn) Započnite simulaciju i s računala pc provjerite dostupnost (naredba ping) poslužitelja server i očitajte TTL vrijednost iz ispisa. Zatim, s poslužitelja server provjerite dostupnost računala pc (naredba ping) i očitajte tu TTL vrijednost iz ispisa. Kojim putem idu paketi u jednom, a kojim putem u drugom slučaju? Ukratko objasnite zašto se međusobno razlikuju?

Odgovor: Možemo vidjeti kad PC šalje ping da se prvi paket vraća s TTL-om 60, a svi ostali s 61. To je zato jer u router6 javi serveru da postoji kraći put nego li preko njega, ali prvotno smo to trebali provjeriti te je zbog toga TTL manji za 1. Nije išao prvotno na router7 nego tek kasnije jer mu ja za default odredište pridodano sučelje od router6.

Međutim kad server šalje ping odnosno odredište je PC onda se direktno ide preko routera 7 jer je tako naznačeno u tablici usmjerenja.

Prilažem sliku izvedbe naredbe ping u oba slučaja.

Slučaj kada PC šalje ping na server

```
[root@pc ~]# ping 10.0.4.10
PING 10.0.4.10 (10.0.4.10): 56 data bytes
64 bytes from 10.0.4.10: icmp_seq=0 ttl=60 time=252.147 ms
64 bytes from 10.0.4.10: icmp_seq=1 ttl=61 time=119.975 ms
64 bytes from 10.0.4.10: icmp_seq=2 ttl=61 time=117.494 ms
^C
--- 10.0.4.10 ping statistics ---
3 packets transmitted, 3 packets received, 0.0% packet loss
round-trip min/avg/max/stddev = 117.494/163.205/252.147/62.899 ms
```

Slučaj kada server šalje ping na PC

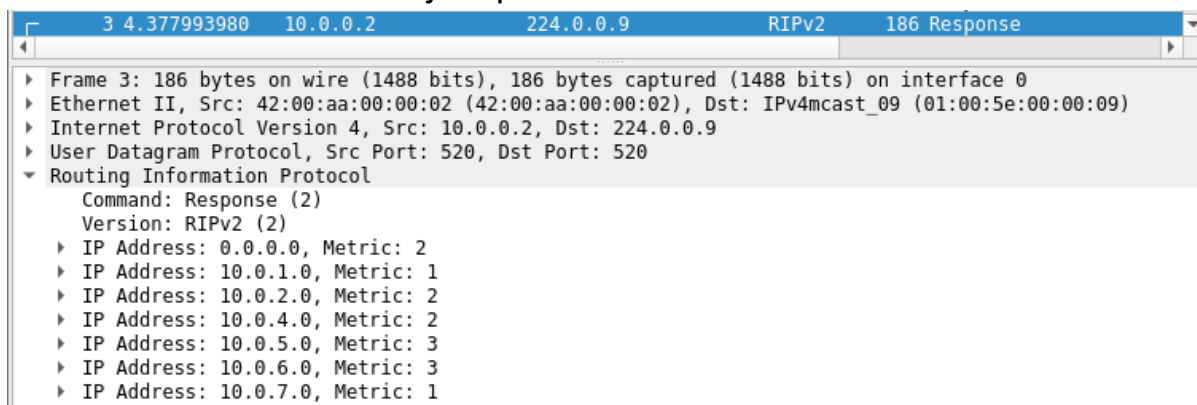
```
IMUNES: server (console) bash
[root@server ~]# ping 10.0.3.20
PING 10.0.3.20 (10.0.3.20): 56 data bytes
64 bytes from 10.0.3.20: icmp_seq=0 ttl=61 time=110.827 ms
64 bytes from 10.0.3.20: icmp_seq=1 ttl=61 time=117.503 ms
64 bytes from 10.0.3.20: icmp_seq=2 ttl=61 time=114.191 ms
^C
--- 10.0.3.20 ping statistics ---
3 packets transmitted, 3 packets received, 0.0% packet loss
round-trip min/avg/max/stddev = 110.827/114.174/117.503/2.725 ms
```

Zadatak 25. (Topologija RIP/RIP.imn) U emulatoru/simulatoru IMUNES, pomoću alata Wireshark snimate paket koji pripada protokolu RIP, proučite njegov sadržaj, te ga ukratko komentirajte.

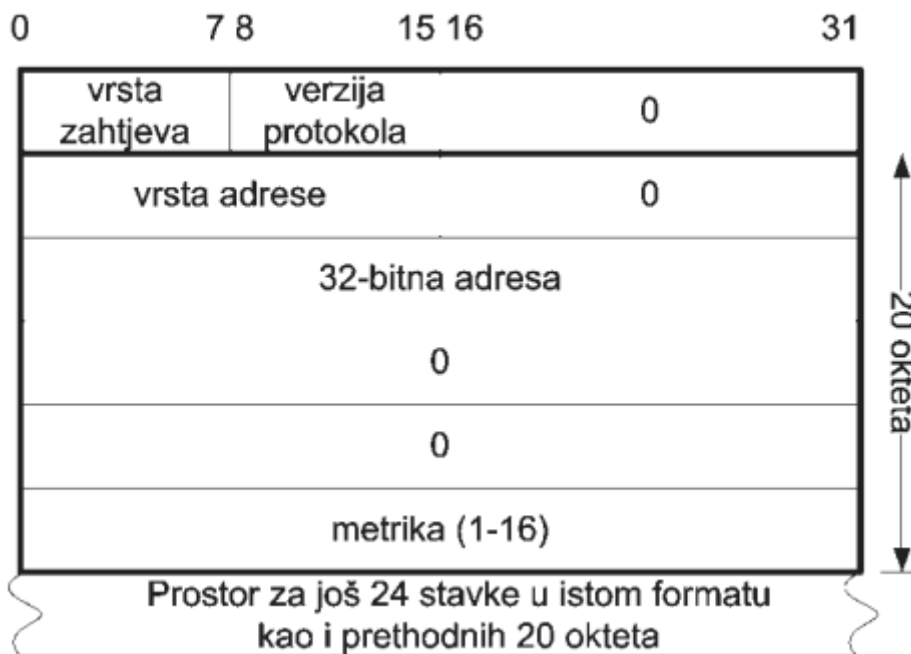
Odgovor: RIP je protokol koji spada u prokole za unutarnje usmjerenje i temelji se na vektorima udaljenosti. U njemu se koristi transportni protokol UDP za prijenos poruka kao što se može vidjeti iz priložene slike paketa. *Command* u snimljenom paketu ima oznaku *Response* što znači da je ovo

RIP odgovor, a *Version* što označava verziju protokola je *RIPv2* odnosno on konstatno promatra mrežu i ako se dogodi nekakva promjena za najkraći put automatski se ažurira tablica usmjerenja. U ovom odgovoru je uz svaku IP adresu u topologiju prikazana oznaka *Metric* odnosno uz nju broj skokova koji je potreban do te podmreže. Format RIPv2 protokola je isti kao format običnog RIP protokola koji je prikazan dolje u nastavku.

Snimljeni paket u alatu Wireshark



Format RIP protokola



Zadatak 28. Učitajte u IMUNES mrežu Ping/ping.imn. Pomoću alata Wireshark snimite proizvoljan TCP-promet koji pripada jednoj vezi te odredite segmente koji se razmjenjuju u fazama uspostave veze i raskida veze (za generiranje TCP-prometa iskoristite alat netcat).

a. Skicirajte razmjenu segmenata za te dvije faze, uz navođenje korištenih TCP-zastavica.

b. Za uhvaćeni promet, odredite koje se adrese i vrata koriste na izvoru i na odredištu. Imaju li svi segmenti istu četvorku {izvorišna IP-adresa, izvorišna vrata, odredišna IP-adresa, odredišna vrata}?

c. Skicirajte razmjenu nekoliko TCP-segmenata u fazi trajanja veze, uz navođenje korištenih TCP-zastavica.

d. Utvrdite na koji se način koriste potvrde u TCP-vezi. Komentirajte.

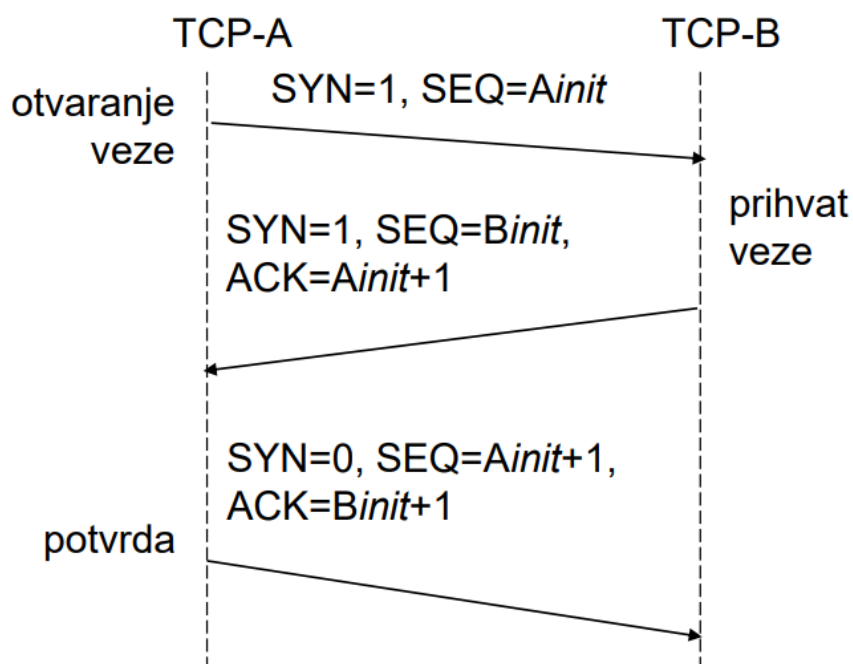
e. Snimite proizvoljan TCP-promet (koji pripada jednoj vezi) i utvrdite veličine prozora. Mijenja li se veličina prozora često u tijeku trajanja TCP-veze? Objasnite.

Odgovor:

- a) Na serveru sam pokrenuo naredbu `nc -l 100`, a na pc1 naredbu `nc 10.0.8.10 100`. Time sam omogućio uspostavljanje veza između pc1 i servera na TCP - vratima 100.

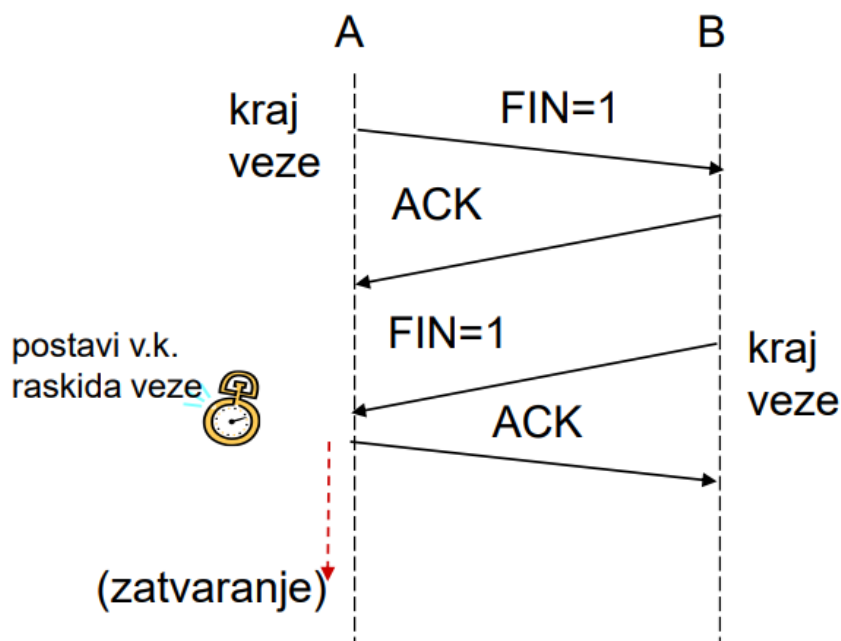
Uspostava veze:

3	23.277797693	10.0.0.21	10.0.8.10	TCP	74	48740 → 100 [SYN] Seq=0 Win=65535 Len=0 MSS=1460 WS
4	23.277890715	10.0.8.10	10.0.0.21	TCP	74	100 → 48740 [SYN, ACK] Seq=0 Ack=1 Win=65535 Len=0
5	23.277896325	10.0.0.21	10.0.8.10	TCP	66	48740 → 100 [ACK] Seq=1 Ack=1 Win=65664 Len=0 TSval



Prekid veze:

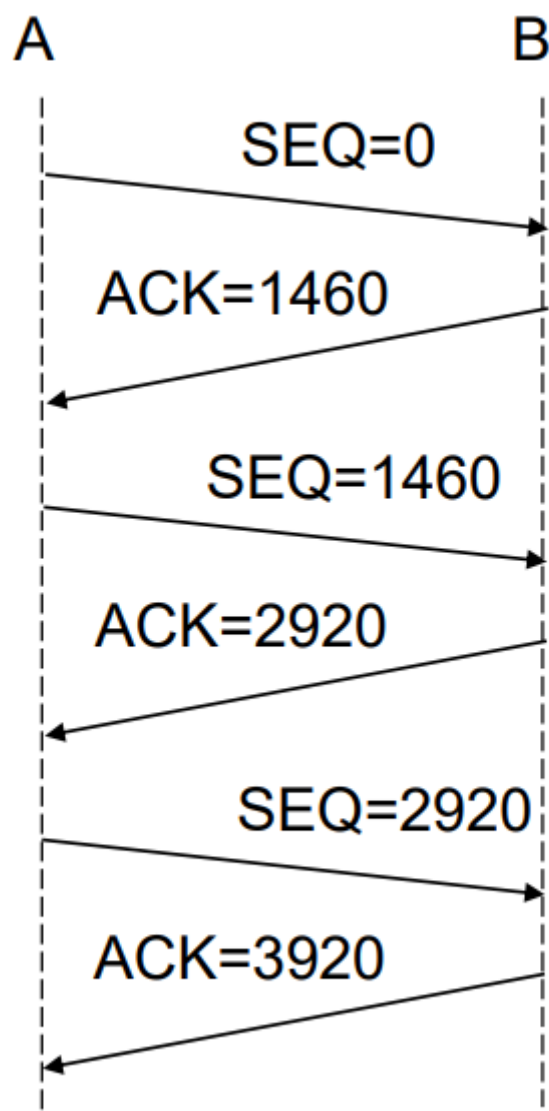
8	40.481464747	10.0.8.10	10.0.0.21	TCP	66 100 → 48740	[FIN, ACK] Seq=1 Ack=1 Win=65564 Len=0
9	40.481510171	10.0.0.21	10.0.8.10	TCP	66 48740 → 100	[ACK] Seq=1 Ack=2 Win=65564 Len=0 TSval=
10	44.190197474	10.0.0.21	10.0.8.10	TCP	66 48740 → 100	[FIN, ACK] Seq=1 Ack=2 Win=65564 Len=0
11	44.190497995	10.0.8.10	10.0.0.21	TCP	66 100 → 48740	[ACK] Seq=2 Ack=2 Win=65564 Len=0 TSval=



- b) Izvorišna IP-adresa je 10.0.0.21 uz vrata 48470, a odredišna IP-adresa je 10.0.8.10 uz vrata 100 što je i jedno od svojstva TCP protokola da svi segmenti imaju istu četvorku.
- c) Prilažem sliku razmjene TCP segmenta u fazi trajanja veze. Može se vidjeti da se koriste zastavice acknowledge i push. Ovo je snimanje prometa pri slanju dva podataka.

2	1.489683618	10.0.0.21	10.0.8.10	TCP	70 52878 → 101	[PSH, ACK] Seq=1 Ack=1 Win=1026 Len=4 T
3	1.583144333	10.0.8.10	10.0.0.21	TCP	66 101 → 52878	[ACK] Seq=1 Ack=5 Win=1026 Len=0 TSval=
4	12.394848994	10.0.0.21	10.0.8.10	TCP	74 52878 → 101	[PSH, ACK] Seq=5 Ack=1 Win=1026 Len=8 T
5	12.494774311	10.0.8.10	10.0.0.21	TCP	66 101 → 52878	[ACK] Seq=1 Ack=13 Win=1026 Len=0 TSval=

Ispod priložena slika prikazuje drugi primjer komunikacije odnosno kako se zastavice SEQ i ACK ponašaju pri prijenosu podataka

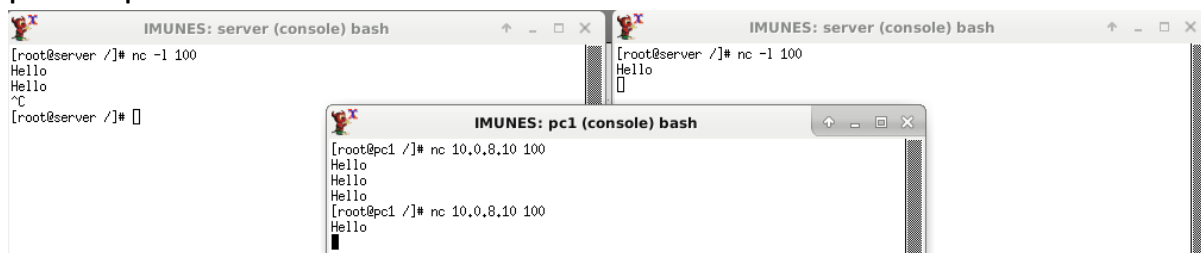


- d) Kako bi se osigurao pouzdan prijenos okteta između procesa, TCP koristi mehanizam potvrđivanja. Ispravan primitak svakog poslanog okteta mora biti potvrđen od strane primatelja. Ukoliko pošiljalatelj ne dobije potvrdu o primitku okteta koje je već poslao, ponavlja se slanje istih okteta sve dok se ne dobije potvrda da ih je odredište primilo. U zaglavlju TCP-a za potvrđivanje se koristi posebno polje ACK. Prije slanja okteta između udaljenih procesa, TCP mora uspostaviti vezu.
- e) Tijekom prozora se veličina prozora nije mijenjala što možemo vidjeti i na gore navedenim slikama zbog toga što prije slanja primatelj definira koliko podataka može primiti te po tome pošiljalatelj šalje toliki broj podataka odnosno tolika je veličina kliznog prozora. Kako nema

zagušenja u mreži nema potrebe za mijenjanjem veličine kliznog prozora

Zadatak 29. (Topologija Ping/ping.imn) Utvrdite mogu li se na jednom računalu pokrenuti dva procesa koji slušaju na istim vratima (npr., pokušajte dvaput pokrenuti alat netcat, istovremeno iz dvije konzole istog računala). Komentirajte.

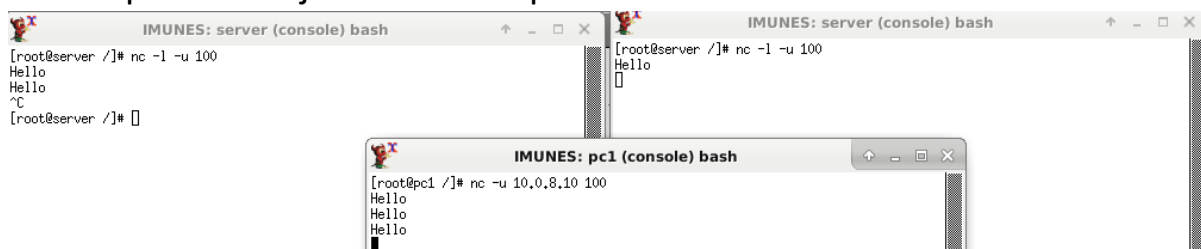
Odgovor: Ne mogu se pokrenuti dva procesa koji slušaju na istim vratim kao što se vidi iz priložene slike gdje se vidi da je u dva različita procesa pokrenuto slušanje nad istim vratima no samo jedan prima poslan poadatak. Tek nakon prekida te veze drugi proces može primiti podatak. To se vidi iz priložene slike gdje nakon gašenja 1. procesa i pokušaja slanja podatka veza se prekida te se ona treba ponovo pokrenuti kako bi 2. proces primio podatak.



The screenshot shows three terminal windows. The top-left window is titled 'IMUNES: server (console) bash' and shows a netcat listener on port 100: '[root@server ~]# nc -l 100', followed by 'Hello' being received twice, and then the user pressing 'C' to exit. The top-right window is also titled 'IMUNES: server (console) bash' and shows the same netcat listener on port 100, with 'Hello' received once. The bottom window is titled 'IMUNES: pc1 (console) bash' and shows a netcat client on IP 10.0.8.10 port 100: '[root@pc1 ~]# nc 10.0.8.10 100', followed by 'Hello' being sent three times.

Zadatak 30. (Topologija Ping/ping.imn) Ukoliko se alatu netcat ne zada protokol koji će koristiti, on podrazumijeva protokol TCP. Ponovite pokus iz prethodnog zadatka uz korištenje protokola UDP te komentirajte

Odgovor: Kod UDP protokola se mogu pokrenuti dva procesa samo što onaj drugi osluškuje promet i tek kad se prvi ugasi drugi može automatski početi primiti podatke bez prekida veze. To se može vidjeti iz priložene slike gdje se Hello šalje triput dok je jedna veza uspostavljena. Nakon 2 puta prekinuli smo 1. proces i 2. je automatski primio treći Hello.



The screenshot shows three terminal windows. The top-left window is titled 'IMUNES: server (console) bash' and shows a netcat listener on port 100 with the UDP flag: '[root@server ~]# nc -l -u 100', followed by 'Hello' being received twice, and then the user pressing 'C' to exit. The top-right window is also titled 'IMUNES: server (console) bash' and shows the same netcat listener on port 100 with the UDP flag, with 'Hello' received once. The bottom window is titled 'IMUNES: pc1 (console) bash' and shows a netcat client on IP 10.0.8.10 port 100 with the UDP flag: '[root@pc1 ~]# nc -u 10.0.8.10 100', followed by 'Hello' being sent three times.

Zadatak 31. (Topologija Ping/ping.imn) Primijetite da, iako su bitno različiti po svojstvima, protokoli UDP i TCP po funkcionalnosti spadaju u transportni sloj referentnog modela OSI. Objasnite zašto.

Odgovor: Unatoč svojim razlikama u garantiranju primitka podatka na odredište i redoslijeda primanja okteta s obzirom na redoslijed kakvim su poslani oba protokola u suštini služe za prijenos podataka. Upravo tome služi transportni sloj koji je odgovaran za prijenos podataka između računala odnosno izvora i odredišta.

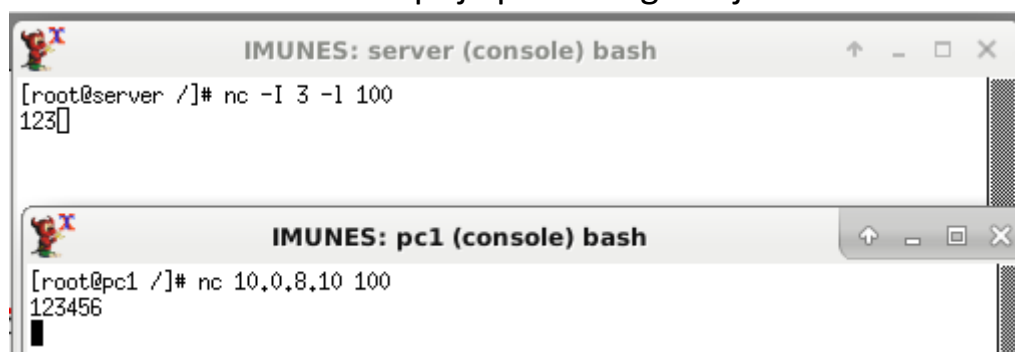
Zadatak 32. (Topologija Ping/ping.imn) Pokušajte prouzročiti gubitak TCP-segmenata. Na koji način možete utvrditi da je došlo do gubitka? Možete li izazvati gubitke paketa bez mijenjanja karakteristika poveznica mreže?

Odgovor: Može se ustrvrditi da je došlo do gubitka tako da se ponovno šalje segment od strane pošiljatelja što možemo vidjeti na priloženoj slici snimke prometa. Može se izazvati pomoću naredbe `-I` (veliko i) pri postavljanju slušanje mreže gdje stavimo da se prvotno prime određeni broj segmenta. Naprimjer naredbom `nc -I 3 -I 100` se događa da kada se pošalje poruka 123456 prvo će se primiti 123 te će kasnije zbog toga što se nije primio cijeli paket on ponovno poslat i bit će primljen. To se vidi na priloženim slikama konzola pc1 i servera. Ovime samo mijenjamo postavke „osluškivanja“ a ne karakteristike poveznice mreže.

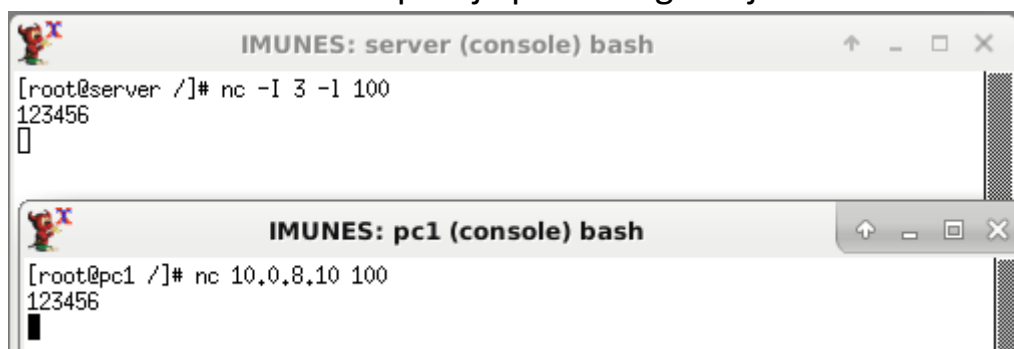
Snimanje prometa pri gubitki segmenta

403	4627.4857344...	10.0.0.21	10.0.8.10	TCP	66	62206 → 100 [FIN, ACK] Seq=15 Ack=1 Win=65664 Len=0 TSval=40645582...
404	4627.4857509...	10.0.8.10	10.0.0.21	TCP	66	100 → 62206 [ACK] Seq=1 Ack=16 Win=16384 Len=0 TSval=3694432891 TS...
405	4627.4858137...	10.0.8.10	10.0.0.21	TCP	66	100 → 62206 [FIN, ACK] Seq=1 Ack=16 Win=16384 Len=0 TSval=36944328...
406	4627.4858666...	10.0.0.21	10.0.8.10	TCP	66	62206 → 100 [ACK] Seq=16 Ack=2 Win=65664 Len=0 TSval=4064558240 TS...
407	4635.2505537...	10.0.0.21	10.0.8.10	TCP	74	52424 → 100 [SYN] Seq=0 Win=65535 Len=0 MSS=1460 WS=64 SACK_PERM=1...
408	4635.2505670...	10.0.8.10	10.0.0.21	TCP	74	100 → 52424 [SYN, ACK] Seq=0 Ack=1 Win=3 Len=0 MSS=1460 WS=64 SACK...
409	4635.2505943...	10.0.0.21	10.0.8.10	TCP	66	52424 → 100 [ACK] Seq=1 Ack=1 Win=65664 Len=0 TSval=3034426563 TSe...
410	4636.0380191...	10.0.8.1	224.0.0.9	RIPv2	186	Response
411	4641.0471964...	10.0.0.21	10.0.8.10	TCP	69	52424 → 100 [ACK] Seq=1 Ack=1 Win=65664 Len=3 TSval=3034432365 TSe...
412	4641.1444253...	10.0.8.10	10.0.0.21	TCP	66	[TCP ZeroWindow] 100 → 52424 [ACK] Seq=1 Ack=4 Win=0 Len=0 TSval=3...
413	4646.1444861...	10.0.0.21	10.0.8.10	TCP	67	[TCP ZeroWindowProbe] 52424 → 100 [ACK] Seq=4 Ack=1 Win=65664 Len=...
414	4646.1444995...	10.0.8.10	10.0.0.21	TCP	66	[TCP ACKed unseen segment] 100 → 52424 [ACK] Seq=1 Ack=5 Win=16384...
415	4646.1445254...	10.0.0.21	10.0.8.10	TCP	69	[TCP Previous segment not captured] 52424 → 100 [PSH, ACK] Seq=5 A...
416	4646.2426907...	10.0.8.10	10.0.0.21	TCP	66	[TCP ACKed unseen segment] 100 → 52424 [ACK] Seq=1 Ack=8 Win=16384...

Konzola prije ponovnog slanja



Konzola poslije ponovnog slanja



Zadatak 33. (Topologija Ping/ping.imn) Pokušajte identificirati promet koji pripada jednoj TCP-vezi za vrijeme u kojem dolazi do gubitka segmenata. Utvrdite što se tada događa s potvrđama i veličinom prozora.

Odgovor: Kao i u prošlom zadatku koristi se ista slika snimanja prometa.

Snimanje prometa pri gubitku segmenta

403	4627.4857344...	10.0.0.21	10.0.8.10	TCP	66	62206 → 100 [FIN, ACK] Seq=15 Ack=1 Win=65664 Len=0 TSval=40645582...
404	4627.4857509...	10.0.8.10	10.0.0.21	TCP	66	100 → 62206 [ACK] Seq=1 Ack=16 Win=16384 Len=0 TSval=3694432891 TS...
405	4627.4858137...	10.0.8.10	10.0.0.21	TCP	66	100 → 62206 [FIN, ACK] Seq=1 Ack=16 Win=16384 Len=0 TSval=36944328...
406	4627.4858666...	10.0.0.21	10.0.8.10	TCP	66	62206 → 100 [ACK] Seq=16 Ack=2 Win=65664 Len=0 TSval=4064558240 TS...
407	4635.2505537...	10.0.0.21	10.0.8.10	TCP	74	52424 → 100 [SYN] Seq=0 Win=65535 Len=0 MSS=1460 WS=64 SACK PERM=1...
408	4635.2505670...	10.0.8.10	10.0.0.21	TCP	74	100 → 52424 [SYN, ACK] Seq=0 Ack=1 Win=3 Len=0 MSS=1460 WS=64 SACK...
409	4635.2505943...	10.0.0.21	10.0.8.10	TCP	66	52424 → 100 [ACK] Seq=1 Ack=1 Win=65664 Len=0 TSval=3034426563 TSe...
410	4636.0380191...	10.0.8.1	224.0.0.9	RIPv2	186	Response
411	4641.0471964...	10.0.0.21	10.0.8.10	TCP	69	52424 → 100 [ACK] Seq=1 Ack=1 Win=65664 Len=3 TSval=3034432365 TSe...
412	4641.1444253...	10.0.8.10	10.0.0.21	TCP	66	[TCP ZeroWindow] 100 → 52424 [ACK] Seq=1 Ack=4 Win=0 Len=0 TSval=3...
413	4646.1444861...	10.0.0.21	10.0.8.10	TCP	67	[TCP ZeroWindowProbe] 52424 → 100 [ACK] Seq=4 Ack=1 Win=65664 Len=...
414	4646.1444995...	10.0.8.10	10.0.0.21	TCP	66	[TCP ACKed unseen segment] 100 → 52424 [ACK] Seq=1 Ack=5 Win=16384...
415	4646.1445254...	10.0.0.21	10.0.8.10	TCP	69	[TCP Previous segment not captured] 52424 → 100 [PSH, ACK] Seq=5 A...
416	4646.2426907...	10.0.8.10	10.0.0.21	TCP	66	[TCP ACKed unseen segment] 100 → 52424 [ACK] Seq=1 Ack=8 Win=16384...

Primatelj šalje poruku TCP ZeroWindow (Win=0) pošiljatelju kao oznaku da nije primio sve segmente sa ACK posljednjeg segmenta. Na to njemu pošiljatelj šalje TCP ZeroWindowProbe (Win=65664) čime prepoznaje koliko je segmenata primljeno odnosno koliko ih nije. Primatelj također šalje TCP ACKed unseen segment (Win=16384) kao oznaku gdje se zaustavilo slanje segmenta. Pošiljatelj nakon toga ponovno šalje segment preko TCP Previous segment not captured (Win=65664) koji primatelj prima te šalju potvrdu pomoću TCP ACKed unseensegment (Win=16384) kao potvrdu primanja ostatka segmenta.